🛒 SmartCart — eCommerce Platform

Technical Documentation v1.0

▶️ 📌 Overview

> A single-vendor eCommerce web application built for independent sellers — offering a clean storefront for customers and a powerful admin panel for product control.

Stack

- Angular (SPA)
- Spring Boot
- Spring Security (JWT)
- MySQL
- JPA / Hibernate

Timeline

- MVP: 2–3 months

▶️ 1️⃣ Goals & Objectives

▶️ 🎯 Business Goals

- Launch MVP within 2–3 months
- Enable complete end-to-end purchase flow
- Provide full product control for admins

▶️ 🛠️ Technical Goals

- Stateless JWT authentication

- RESTful API (resource-oriented design)
- Modular Monolith backend
- Scalable & microservice-ready structure
- Clean Angular component architecture

▶️ 2️⃣ Target Users

▶️ 👤 Customer

Needs

- Browse products
- Search products
- Add to cart
- Checkout securely
- View order history

▶️ 👨‍💼 Admin

Needs

- Add products
- Update product details
- Delete products
- Monitor orders

▶️ 3️⃣ User Stories

▶️ 🛍️ Customer Stories

- As a customer, I want to browse products so that I can see available items.
- As a customer, I want to search products to find specific items.
- As a customer, I want to add items to cart so I can purchase multiple products.

- As a customer, I want secure checkout to complete my purchase.

▶️ ⚙️ Admin Stories

- As an admin, I want to add new products so they appear in the store.
- As an admin, I want to update product details so information stays accurate.
- As an admin, I want to delete products when unavailable.

▶️ 4️⃣ Functional Requirements

▶️ 🔐 Authentication Module

- POST `/register` — Public
- POST `/login` — Public (returns JWT)

▶️ 📦 Product Module

- GET `/products` — Public (paginated)
- POST `/products` — Admin
- PUT `/products/{id}` — Admin
- DELETE `/products/{id}` — Admin

▶️ 🛒 Cart Module

- POST `/cart/add` — Authenticated
- DELETE `/cart/{itemId}` — Authenticated
- PATCH `/cart/{itemId}` — Authenticated

▶️ 💳 Checkout Module

- POST `/orders` — Authenticated
  - Creates order record
  - Clears cart

▶️ 5️⃣ Non-Functional Requirements

▶️ ⚡ Performance

- Page load < 2 seconds
- API response < 500ms
- Pagination enforced

▶️ 🔐 Security

- BCrypt password hashing
- JWT expiration & validation
- Input validation (Bean Validation)
- Prepared statements (SQL injection protection)

▶️ 📱 Usability

- 100% responsive Angular frontend
- Mobile-first design

▶️ 🪵 Observability

- Structured logging
- Consistent API error responses

▶️ 6️⃣ Technology Stack

▶️ ☕ Backend

- Java 17 (LTS)
- Spring Boot
- Spring Security (JWT)
- JPA / Hibernate
- MySQL (3NF normalized schema)

▶️ 🅰️ Frontend

- Angular (SPA)
- Angular HttpClient

▶️ ⚙️ Configuration Example

```
spring.datasource.url=jdbc:mysql://localhost/smartcart
spring.jpa.hibernate.ddl-auto=update

jwt.expiration=86400000
jwt.secret=your-secret-key

server.port=8080
```

▶️ 7️⃣ Architecture

▶️ 🏗️ Architectural Style

Modular Monolith

Single deployable application divided into bounded internal modules.

Ready for future microservices extraction.

▶️ 🖥️ Layered Structure

Client Layer

- Angular SPA

⬇️ HTTPS / REST

API Layer

- Spring Boot Controllers
- Spring Security (JWT Filter Layer)

⬇️

Business Modules

- User Module
- Product Module
- Cart Module
- Order Module
- Admin Module

⬇️ JPA

Data Layer

- MySQL Database

▶️ 🧠 Module Responsibilities

▶️ User Module

- Authentication
- Role management (ADMIN / CUSTOMER)
- JWT issuance

▶️ Product Module

- CRUD operations
- Pagination
- Indexed search

▶️ Cart Module

- Per-user cart
- Quantity management
- Real-time total calculation

▶️ Order Module

- Order creation
- Order item persistence

- Status lifecycle tracking

▶️ Admin Module

- Product management
- Order monitoring

▶️ 8️⃣ Database Design

▶️ 📊 Schema Strategy

- Fully normalized (3NF)
- Indexed columns:
  - `product.name`
  - `product.category`

▶️ 🔗 Relationships

- User 1:1 Cart
- Cart 1:N CartItem
- CartItem N:1 Product
- User 1:N Order
- Order 1:N OrderItem
- OrderItem N:1 Product

▶️ 9️⃣ Performance Optimization

▶️ 🧮 Pagination

Spring `Pageable` used for product listing.

▶️ 🔍 Indexing

Indexes on product name & category.

▶️ 💤 Lazy Loading

JPA relationships fetched on demand.

▶️ 🚫 N+1 Prevention

Use `JOIN FETCH` in JPQL when retrieving related entities.

▶️ 🔟 Scalability Roadmap

▶️ 🔀 Microservices Migration

Modules can be extracted into:

- User Service
- Product Service
- Order Service

▶️ ⚡ Redis Caching

Cache product catalog & search results.

▶️ 💰 Payment Gateway

Stripe / PayPal integration.

▶️ 🌐 CDN Integration

Offload product images to CloudFront or Cloudflare.

▶️ 1️⃣1️⃣ Risks & Mitigation

▶️ 🔴 High Risks

- Stock concurrency → Use optimistic locking
- Security misconfiguration → Security audit & environment secrets

▶️ 🟡 Medium Risks

- Cart state inconsistency → Server-side cart

- Large catalog performance → Indexes + enforced pagination

▶️ 1️⃣2️⃣ Success Metrics

▶️ 📈 Product Metrics

- Order success rate
- Registered user growth

▶️ ⚙️ Operational Metrics

- API < 500ms
- Zero critical admin failures
- Zero critical CVEs